

Title:	FreeRTOS on the Cortex M3
Author:	Craig Duffy 07/12/14, 15/15/18, 22/02/22
Module:	Secure Embedded Systems
Awards:	BSc CSI, BSc Forensic Computing, Computer Security.
Prerequisites:	Experience programming the Olimex board, ARM Cortex architecture and operating systems knowledge

We have seen with our programs to date that to use the resources of the Olimex board we require more and more sophisticated software. In order to really exploit the board's features something like an operating system is needed. We don't have enough memory or power to run a system as large as the Microsoft Windows operating systems or Linux, even the stripped down, MMUless version, uClinux. However the use of an operating system would allow us to use the full resources of the board in an efficient way.

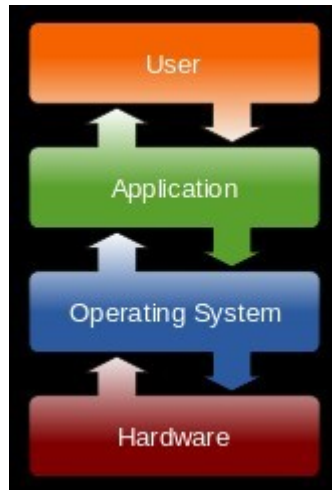


Figure 1 – locating the operating system.

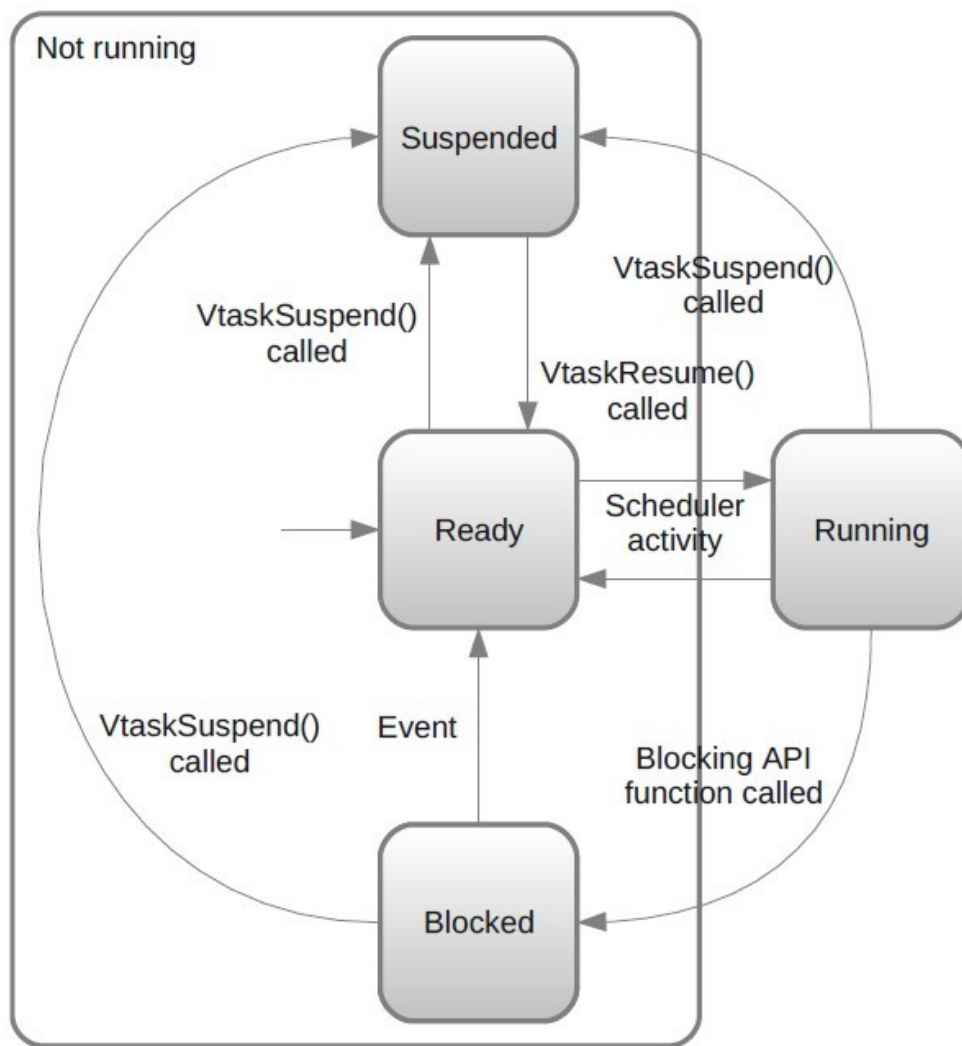
Operating Systems

There are a few different types of operating systems. The ones you are probably most familiar with are general purpose operating systems such as Windows and Linux. There are also specialist OSs for specific purposes for example systems which require very strict time deadlines are called Real Time Operating Systems. Generally the more hardware resources there are the more features and functions the system will provide. In our limited system we only really want to provide the basic features, that is, to be able to run tasks and facilitate their communication. This type of operating system is called an **executive**, as its major task is to execute tasks. The executive we are looking at is called **FreeRTOS**, which is a slight misnomer in that it is more an executive than OS!

Tasks, processes and threads

We have already stated that an executive runs tasks but what is a task? Generally speaking a task is some set of work that the computer requires doing and the resources required to undertake that work. Sadly this answer isn't entirely without problems. In computing there are a number of overlapping terms which could fit the bill – process, thread and task. Process is the most generic and describes what happens when you execute a program or application on a general purpose computer. Threads are subsets of processes, normally you would expect to have a number of threads within one process. Finally task is somewhere between the two previous descriptions, a sort of light-weight process. It is worth noting that task is the word used in the FreeRTOS documentation.

FreeRTOS therefore is basically a system that allows us to go through a cycle of creating tasks, and all the resources they need, execute and suspend them and to allow multiple tasks to interact in a controlled and sensible way. This means that the scheduler must be able to decide when to run a task, when to terminate it, and what to do with its resources upon termination. Now if the system only had one task life would be simple, but as we know hardware has lots of peripherals and to be able to have multiple tasks accessing these resources would be beneficial. So FreeRTOS must be able to know what order to run tasks in, and what to do when more than one task is requesting the CPU. Below is a diagram of the various states that a FreeRTOS task can be in and the various calls to manage the state transitions that are supplied within the system.



FreeRTOS task states

Primitives

FreeRTOS has to manage the tasks and control them, in order to do this it supplies a number of central primitives or system calls to facilitate this. A central core of this is the ability to queue tasks and select them from the various queues – Blocked, Ready to Run, Suspended etc. In order to control access to these queues a number of control mechanisms, semaphores and mutexes, are provide to support exclusive access to the resources.

Getting FreeRTOS

The FreeRTOS source code can be downloaded from the following sourceforge site:

<http://sourceforge.net/projects/freertos/files/latest/download?source=files>

This will give the latest zipped version – version V202012.00 at the time of writing. You may find that the version you download is newer, but the instructions in the worksheet should still apply. If in doubt you can download version V202012.00 and then upgrade later. It is best to upgrade eventually as if you do ask for help online people won't help fix problems in outdated versions. In the rest of the worksheet the listings are from a previous version of FreeRTOS so some of the images won't display everything you will see, but the important files and directories will be as presented.

Files

```
craig@craig-HP-Compaq-8200-Elite-SFF-PC:~/stm32/M3/FreeRTOSV8.2.3$ tree -L 1 -d
.
├── FreeRTOS
└── FreeRTOS-Plus

2 directories
craig@craig-HP-Compaq-8200-Elite-SFF-PC:~/stm32/M3/FreeRTOSV8.2.3$
```

FreeRTOS top level directory

From the top level there are 2 directories – FreeRTOS which has the source in it and FreeRTOS-Plus which has a number of implementations of things such as TCP/IP, file systems and so on. At the moment we are only interested in the FreeRTOS directory.

```
craig@craig-HP-Compaq-8200-Elite-SFF-PC:/tmp/FreeRTOS$ tree -L 1 -d
.
├── Demo
├── License
└── Source

3 directories
craig@craig-HP-Compaq-8200-Elite-SFF-PC:/tmp/FreeRTOS$
```

The FreeRTOS subdirectory

The FreeRTOS directory has 3 sub-directories – Demo, License and Source. Demo is filled with a large number of specific implementations for a wide variety of target boards, CPUs, and tool chains. There is an implementation of FreeRTOS for our CortexM3 STM32F107. Normally one would take such an implementation and use it as the basis for our own board. We are not taking this approach for a number of reasons. This tends to lead to a complex implementation with lots of code that you are unsure of and quite often leads to difficulty in solving bugs later on. The Demo implementation also uses the Rowley toolchain and IDE which we aren't using, so it can take a fair amount of shoe-horning. Finally in this worksheet I am going to give a guided port so we don't to follow anyone else's example.

The License directory contains all of the legal and licensing information – it uses a standard GNU GPL license. The Source directory contains the source code for FreeRTOS. A truncated listing of the contents are given below. The source code for the systems is pretty small, compared with Linux or OSX for example. There are 3 main files – task.c, queue.c and list.c which make the main core of FreeRTOS. The directory portable, which has been truncated in our output to just show the ARM Cortex M3 and GCC directories, contains the code that requires machine specific code to run. Included in this directory are the memory allocation routines – 5 variants at time of writing, which are available for the system.

```
craig@craig-HP-Compaq-8200-Elite-SFF-PC:/tmp/FreeRTOS$ tree Source/
Source/
├── croutine.c
├── event_groups.c
├── include
│   ├── croutine.h
│   ├── deprecated_definitions.h
│   ├── event_groups.h
│   ├── FreeRTOS.h
│   ├── list.h
│   ├── mpu_wrappers.h
│   ├── portable.h
│   ├── projdefs.h
│   ├── queue.h
│   ├── semphr.h
│   ├── StackMacros.h
│   ├── stdint.readme
│   ├── task.h
│   └── timers.h
├── list.c
├── portable
│   ├── GCC
│   │   ├── ARM_CM3
│   │   │   ├── port.c
│   │   │   └── portmacro.h
│   │   ├── ARM_CM3_MPU
│   │   │   ├── port.c
│   │   │   └── portmacro.h
│   └── MemMang
│       ├── heap_1.c
│       ├── heap_2.c
│       ├── heap_3.c
│       ├── heap_4.c
│       └── heap_5.c
├── readme.txt
├── queue.c
├── readme.txt
├── tasks.c
└── timers.c

6 directories, 31 files
craig@craig-HP-Compaq-8200-Elite-SFF-PC:/tmp/FreeRTOS$
```

FreeRTOS Source directory

The include directory has most of the header files the system needs. You will notice in our untarred version of **FreeRTOS_introduction** that the **Makefile** has to point to these source files and directories.

```
#FreeRTOS object files
RTOSOBJ= tasks.o queue.o list.o timers.o heap_1.o port.o
...

#FreeRTOS path
FreeRTOS=../FreeRTOSv202012.00/FreeRTOS/Source
...
vpath %.c $(DEVICE)

#FreeRTOS search paths
vpath %.c $(FreeRTOS)/
```

```
vpath %.c $(FreeRTOS)/portable/MemMang
vpath %.c $(FreeRTOS)/portable/GCC/ARM_CM3
```

FreeRTOS.h

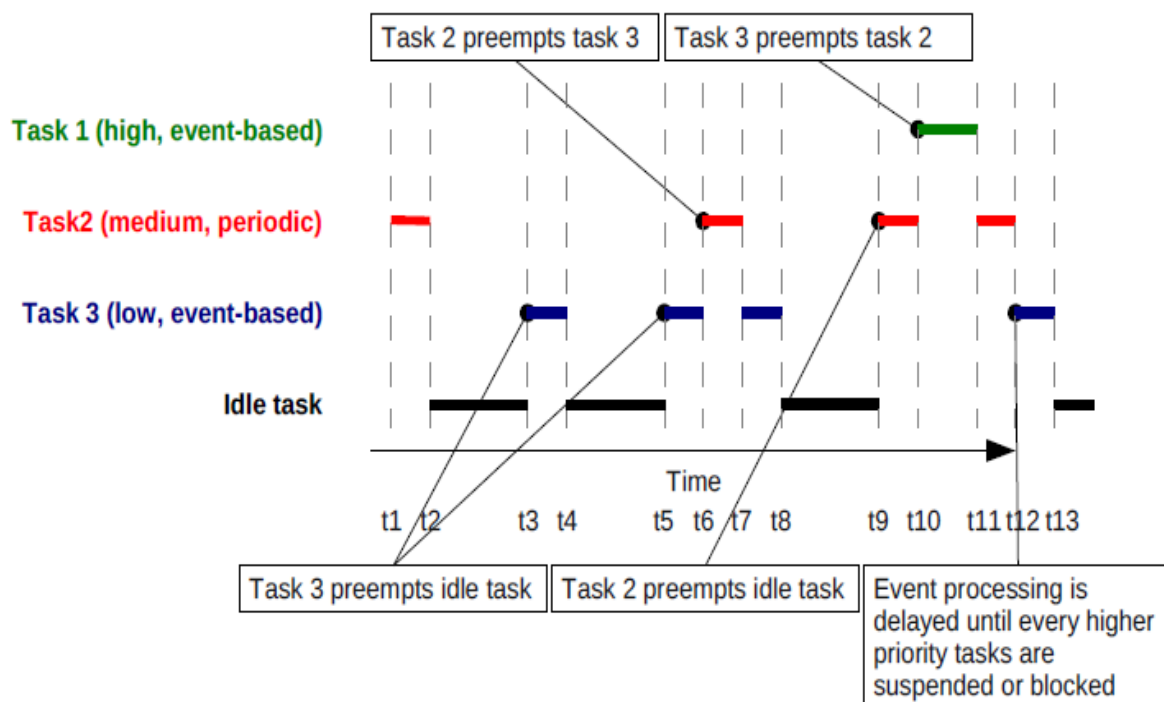
A very important file is the FreeRTOS.h header file, in the worksheet directory. This contains all of the configuration for the system and essentially holds the whole system together. This file holds a series of defines and macros that tell the system what to expect. For example, the FreeRTOS system can use 2 different modules for its scheduler – a cooperative one where tasks give up the CPU voluntarily and a pre-emptive one where the tasks are automatically rescheduled by clock tick. To set a pre-emptive scheduler, the type we will use, make sure you have the following in your **FreeRTOSconfig.h** file.

```
85 | #define configUSE_PREEMPTION 1
```

In most cases 1 means use and 0 means don't use. For example to use the idle tasks then set:

```
86 | #define configUSE_IDLE_HOOK 1
```

It is worth noting that these defines don't create the code, that still needs to be done and the system can't build if it isn't provided. It is normal to include hardware specific low-power/power-down code in the idle task as this is called when the scheduler has run out of tasks to run.



As can be seen from the above diagram the scheduler can use priorities to decide which task to run next. The exact details of how the priorities are handled will be architecture dependant. The Cortex M3 supports 256 levels of priority but our implementation, the ST Micro STM32107 only implements 5 bits of this to give 16 levels of priority, quite enough for our modest needs.

FreeRTOS needs to be able to take control of the CPU in order to run tasks and it achieves this through a series of calls to Cortex M3 interrupt service routines. These are:

```
96 | #define vPortSVCHandler    SVC_Handler
97 | #define xPortPendSVHandler PendSV_Handler
98 | #define xPortSysTickHandler SysTick_Handler
```

This hooks FreeRTOS into the Cortex and allows FreeRTOS to be called automatically. The SysTick hook calls the scheduler on each systems tick, and calls PendSV_Handler to reschedule the tasks. PendSV_Handler is the mechanism for rescheduling tasks, this is separate from a systick to all processes to reschedule themselves independent of the system tick. SVC_Handler is used to implement system calls.

This define is very important as it tells FreeRTOS what the clock rate of the CPU is. If this is wrong then all of the systems timing will go awry. However it is only informing FreeRTOS of the clock rate not setting it. Unlike systems like Windows and Linux, FreeRTOS doesn't expect there to be a BIOS to set up certain system parameters and initialisations and expects the developer to provide those. In that we already have the system start up code in startup_stm32f10x.c/o system_stm32f10x.c/o, we know we can do this already.

```
88 | #define configCPU_CLOCK_HZ      ( ( unsigned long ) 72000000 )
```

The tick rate is also important as this defines when the scheduler is going to kick in. This defines the rate in Hertz, so 1000 means that the systick will start the scheduler every millisecond.

```
89 | #define configTICK_RATE_HZ      ( ( TickType_t ) 1000 )
```

There are some cases where it isn't simply a binary yes/no choice – with the stack overflow check there are 3 possibilities. Stack checking is very important for operating systems like FreeRTOS as it has to use a fixed amount of memory per task and if the stack overflows then this can be very difficult to debug.

```
105 | #define configCHECK_FOR_STACK_OVERFLOW 2
```

The 3 values for CHECK_FOR_STACK_OVERFLOW are:

- 0 – no check – this is dangerous but will speed up context switches and reduce the code size.
- 1 – run the FreeRTOS built in stack checking on each context switch. This will take some small amount of time but it only checks at context switch time and can't detect errors at other times.
- 2 – Do extra checks on top of scenario 1. This writes a pattern to the end of the stack and checks to see if that pattern has been disturbed. This takes slightly longer but is much better at checking for potential stack overflow errors.

The FreeRTOS demonstration projects

In the [FreeRTOS introduction](#) directory there is a very simple demonstration project. This simply creates 2 tasks which flash the 2 LEDS (what else!).

The tasks (in main.c) are very simple and are given below.

```
void vGreenToggleTask(void *pvParameters)
{
    int i;
```



```

    for(;;)
    {
        vParTestToggleLED(0);
        vTaskDelay( 3000 / portTICK_PERIOD_MS );
    }
}

void vOrangeToggleTask(void *pvParameters)
{
    for(;;)
    {
        vParTestToggleLED(1);
        vTaskDelay( 1500 / portTICK_PERIOD_MS );
    }
}

```

The demonstrations tasks

You will notice that the tasks are infinite loops as FreeRTOS doesn't expect them to terminate, although they can be destroyed by a `vTaskDelete()` call. The differences between them are small – they both call functions from the `partest.c` file which toggle the LEDs – and each function picks a different LED to flash, also (just to show the difference) the calls to the `vTaskDelay` function are slightly different. The `vTaskDelay` function passes the number of milliseconds that the task is to be suspended for, this is better than using a polling loop as `vTaskDelay()` will take the task from the running state and allow the rescheduling of another task.

In order to run the tasks the scheduler must be called. This is called after all the hardware initialisation has been done. This code is also found in `main.c`,

```

int i,j=0;
vParTestInitialise();
xTaskCreate( vGreenToggleTask, "GreenLedToggle", 128, NULL,
             mainGREEN_LED_TOGGLE, NULL );
for(i=0;i != 100000; i++) j++;
xTaskCreate( vOrangeToggleTask, "OrangeLedToggle", 128, NULL,
             mainORANGE_LED_TOGGLE, NULL );

vTaskStartScheduler();

```

The function used to create the tasks is `xTaskCreate`, this has a large number of parameters.

```

portBASE_TYPE xTaskCreate( pdTASK_CODE pvTaskCode,
                           const signed portCHAR * const pcName,
                           unsigned portSHORT usStackDepth,
                           void *pvParameters,
                           unsigned portBASE_TYPE uxPriority,
                           xTaskHandle *pxCreatedTask
);

```

The parameters are:

pvTaskCode – this is the actual code which is to be implemented.

pcName – this is for debugging purposes only and is of no use to FreeRTOS

usStackDepth – this is the size of the stack for the process – this is given in words, so the actual stack size will be architecture dependant. On the Cortex M3, for example a value of 100 will result in a stack size of 400 – as a word is 32 bits or 4 bytes in that architecture.

pvParameters – is the list of arguments to the task. It is good practise to create a special structure to hold the parameters and point to that. If there are no parameters then this can be left NULL.

uxPriority – this is the priority of the task and will affect how it is scheduled. It can be in a range of 0 to MAX_PRIORITIES-1. This is architecture dependant – our implementation of the Cortex core has 16 levels of priority,

pxCreatedTask – this is a handle to the task which allows changes to be made to it at run time. If the task doesn't require handling it can be left as NULL.

Care has to be taken with the stack depth as this can cause very difficult to debug run time errors. Also the use of priorities needs careful thought as it will affect when tasks are executed. A task with too low a priority will end up being starved of CPU time, one on too high a priority can end up taking too much CPU time.

Once the tasks have been created then it is a matter of starting the scheduler to execute the tasks.

Using the repo link or the worksheet8.tgz files, plus the unpacked FreeRTOS source and your ST standard peripheral library code (newlib-arm-none-eabi) try the following exercises.

Exercises.

Pass exercises.

Using your downloaded FreeRTOS files and your set-up for the standard peripheral library build, download and execute the FreeRTOS system. Change the speeds at which the LEDs blink.

Change the priorities of the tasks to create a position of starvation for one task – discuss with your lab tutor how you achieve this.

Add 2 more tasks for the buttons- get the LED tasks and button tasks to use shared variables to decide whether or not to turn on or off the LEDs.

Credit exercises.

The 2 LED tasks are essentially the same code with 2 different parameters – the number of the LED and the delay time. Create one function which can accept these two parameters and then create two different tasks which flash the LEDs at differing rates and with different priorities.

Rewrite the maths game from worksheet6. Make the maths test run as a separate task, have a clock task displaying the time in the top corner of the window. Also you can introduce a count down timer to give the user a limited time span in which complete the puzzle – see worksheet 7. That timer should run as a separate task.

NOTE:

The Makefile in the FreeRTOS_introduction directory is designed to compile and link the FreeRTOS files with the demonstration files. It assumes you are using a certain directory structure and certain versions of the software. You will need to change some variables to allow for your directory structure and code versions.

We have already mentioned that you may have a newer version of FreeRTOS so will need to change the following line:

```
#FreeRTOS path  
FreeRTOS = ../FreeRTOSv202012.00/FreeRTOS/Source
```

Depending on the location of the newlib-arm-none-eabi directory relative to the directory FreeRTOS_introduction you may need to change:

```
# Compilation Flags  
LDLIBS+= ../../newlib-arm-none-eabi/arm-none-eabi/newlib/libc.a  
...  
LDLIBS+= /usr/lib/gcc/arm-none-eabi/9.2.1/libgcc.a  
...  
CFLAGS+= -I../../newlib-arm-none-eabi/arm-none-eabi/newlib/libc/include
```

Also the libgcc.a file may be somewhere else dependant on which compiler has been installed. Other changes may be needed that I haven't thought of!